

ADVANCED RAG (CHUNKING, RERANKING, HYBRID SEARCH)

ACT ON THIS LESSON

Use this loop to turn reading into evidence. Each checkpoint is yours to mark and stays separate from quiz correctness.

0 OF 5 CHECKPOINTS · QUIZ NOT YET PASSED

01

READ

Understand the claim and constraints.

02

BUILD

Create or modify the lesson artifact.

03

RUN

Execute the relevant program or exercise.

04

PROVE

Capture output that supports the claim.

05

CONTINUE

Mark the lesson complete when you are ready.

≡ LESSONS

LOCAL PROGRESS

AI DATA SYSTEMS

PATH 915 MIN

LESSON 90 MIN

RUN FROM THE REPOSITORY

```
python3 phases/11-llm-engineering/07-advanced-rag/code/main.py
```

COPY COMMAND

Run from the repository root, the folder containing README.md.

FOCUSED PATH CONTEXT

Basic RAG retrieves the top-k most similar chunks. That works for simple questions. It falls apart for multi-hop reasoning, ambiguous queries, and large corpora. Advanced RAG is the difference between a demo that works on 10 documents and a system that works on 10 million.

Type: Build

Languages: Python

Prerequisites: Phase 11, Lesson 06 (RAG)

Time: ~90 minutes

Related: Phase 5 · 23 (Chunking Strategies for RAG) covers all six chunking algorithms — recursive, semantic, sentence, parent-document, late chunking, contextual retrieval — with Vectara/Anthropic benchmarks. This lesson builds on top: hybrid search, reranking, query transformation.



LEARNING OBJECTIVES

Implement advanced chunking strategies (semantic, recursive, parent-child) that preserve document structure and context

Build a hybrid search pipeline combining BM25 keyword matching with semantic vector search and a cross-encoder reranker

Apply query transformation techniques (HyDE, multi-query, step-back) to improve retrieval on ambiguous or complex questions

Diagnose and fix common RAG failures: wrong chunk retrieved, answer not in context, multi-hop reasoning breakdown

THE PROBLEM

You built a basic RAG pipeline in Lesson 06. It works for straightforward questions on a small corpus. Now try these:

Ambiguous query: "What was revenue last quarter?" Semantic search returns chunks about revenue strategy, revenue projections, and the CFO's thoughts on revenue growth. All semantically similar to the word "revenue." None containing the actual number. The correct chunk says "\$47.2M in Q3 2025" but uses the word "earnings" instead of "revenue." The embedding model thinks "revenue strategy" is closer to the query than "Q3 earnings were \$47.2M."

Multi-hop question: "Which team had the highest customer satisfaction score improvement?" This requires finding the satisfaction scores for each team, comparing them, and identifying the maximum. No single chunk contains the answer. The information is scattered across team reports.

Large corpus problem: You have 2 million chunks. The correct answer is in chunk #1,847,293. Your top-5 retrieval pulls chunks #14, #89,201, #1,200,000, #44, and #901,333. Close in embedding space, but none containing the answer. At this scale, approximate nearest neighbor search introduces enough error that relevant results get pushed out of the top-k.

Basic RAG fails because vector similarity is not the same as relevance. A chunk can be semantically similar to a query without being useful for answering it. Advanced RAG addresses this with four techniques: hybrid search (add keyword matching), reranking (score candidates more carefully), query transformation (fix the query before searching), and better chunking (retrieve at the right granularity).

✓ PRE-LESSON CHECK

1. What is the limitation of basic top-k semantic search in RAG?

- ☐ It requires GPU
- ☐ It retrieves chunks that are semantically similar to the query but may not contain the actual answer, especially for ambiguous or multi-hop questions
- ☐ It's too slow
- ☐ It can't handle large documents

2. What is hybrid search in the context of RAG?

- ☐ Combining BM25 keyword matching with semantic vector search to capture both exact terms and meaning-based relevance
- ☐ Using both CPU and GPU for search
- ☐ Searching across multiple databases
- ☐ Using two different LLMs

THE CONCEPT

HYBRID SEARCH: SEMANTIC + KEYWORD

Semantic search (vector similarity) is good at understanding meaning. "How do I cancel my subscription?" matches "Steps to terminate your plan" even though they share no words. But it misses exact matches. "Error code E-4021" might not match a chunk containing "E-4021" if the embedding model treats it as noise.

Keyword search (BM25) is the opposite. It excels at exact matches. "E-4021" matches perfectly. But "cancel my subscription" returns zero results if the document says "terminate your plan."

Hybrid search runs both, then merges the results.

BM25 (Best Matching 25) is the standard keyword search algorithm. It has been the backbone of search engines since the 1990s. The formula:

COPY

$$\text{BM25}(q, d) = \sum \text{over terms } t \text{ in } q: \text{IDF}(t) * (\text{tf}(t, d) * (k_1 + 1)) / (\text{tf}(t, d) + k_1 * (1 - b + b * |d| / \text{avgdl}))$$

Where $\text{tf}(t, d)$ is the term frequency of t in document d , $\text{IDF}(t)$ is the inverse document frequency, $|d|$ is the document length, avgdl is the average document length, k_1 controls term frequency saturation (default 1.2), and b controls length normalization (default 0.75).

In plain terms: BM25 scores documents higher when they contain query terms (especially rare ones), but with diminishing returns for repeated terms. A document with the word "revenue" 50 times is not 50x more relevant than one with it once.

RECIPROCAL RANK FUSION (RRF)

You have two ranked lists: one from vector search, one from BM25. How do you combine them? Reciprocal Rank Fusion is the standard approach.

COPY

$$\text{RRF_score}(d) = \sum \text{over rankings } R: 1 / (k + \text{rank_R}(d))$$

Where k is a constant (typically 60) that prevents the top-ranked result from dominating.

A document ranked #1 in vector search and #5 in BM25 gets: $1/(60+1) + 1/(60+5) = 0.0164 + 0.0154 = 0.0318$

A document ranked #3 in vector search and #2 in BM25 gets: $1/(60+3) + 1/(60+2) = 0.0159 + 0.0161 = 0.0320$

RRF naturally balances the two signals. A document that ranks highly in both lists gets the best score. A document that ranks #1 in one list but is absent from the other gets a moderate score. This is robust because it uses ranks, not raw scores, so differences in score distributions between the two systems do not matter.

RERANKING

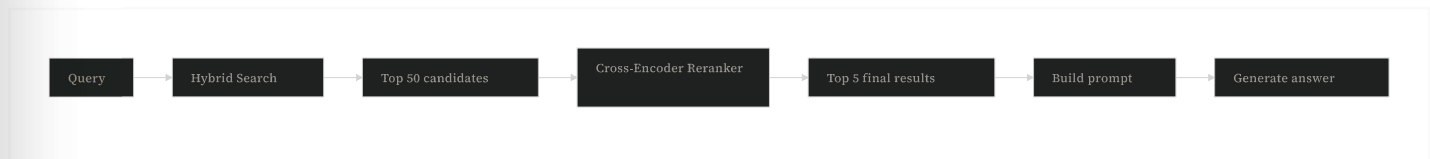
Retrieval (whether vector, keyword, or hybrid) is fast but imprecise. It uses bi-encoders: the query and each document are embedded independently, then compared. The embeddings are

computed once and cached. This scales to millions of documents.

Reranking uses cross-encoders: the query and a candidate document are fed together into a model that outputs a relevance score. The model sees both texts simultaneously and can capture fine-grained interactions between them. A cross-encoder can understand that "What were Q3 earnings?" is highly relevant to a chunk containing "\$47.2M in Q3" even if a bi-encoder missed the connection.

The trade-off: cross-encoders are 100-1000x slower than bi-encoders because they process the query-document pair jointly. You cannot pre-compute cross-encoder scores for a million documents. The solution: retrieve a larger candidate set (top-50 from hybrid search), then rerank with a cross-encoder to get the final top-5.

EXPAND



Common reranking models (2026 lineup):

Cohere Rerank 3.5: managed API, multilingual, best recall gain on mixed corpora

Voyage rerank-2.5: managed API, lowest latency of the hosted options

Jina-Reranker-v2 Multilingual: open-weight, 100+ languages

bge-reranker-v2-m3: open-weight, strong baseline

cross-encoder/ms-marco-MiniLM-L-6-v2: open-weight, runs on CPU for prototyping

ColBERTv2 / Jina-ColBERT-v2: late-interaction multi-vector rerankers — $O(\text{tokens})$ not $O(\text{docs})$ at scoring time

QUERY TRANSFORMATION

Sometimes the problem is not retrieval but the query itself. "What was that thing about the new policy change?" is a terrible search query. It contains no specific terms. The embedding is vague. No retrieval system can find the right documents from this.

Query rewriting: rephrase the user's query into a better search query. An LLM can do this:

COPY

User: "What was that thing about the new policy change?"

Rewritten: "Recent policy changes and updates"

HyDE (Hypothetical Document Embeddings): instead of searching with the query, generate a hypothetical answer, embed that, and search for similar real documents.

COPY

Query: "What is the refund policy for enterprise?"

Hypothetical answer: "Enterprise customers are eligible for a full refund within 60 days of purchase. Refunds are pro-rated based on the remaining subscription period and processed within 5-7 business days."

Embed the hypothetical answer and search for real documents similar to it. The intuition: the hypothetical answer lives closer in embedding space to the real answer than the original question does. Questions and answers have different linguistic structures. By generating a hypothetical answer, you bridge the gap between "question space" and "answer space" in the embedding.

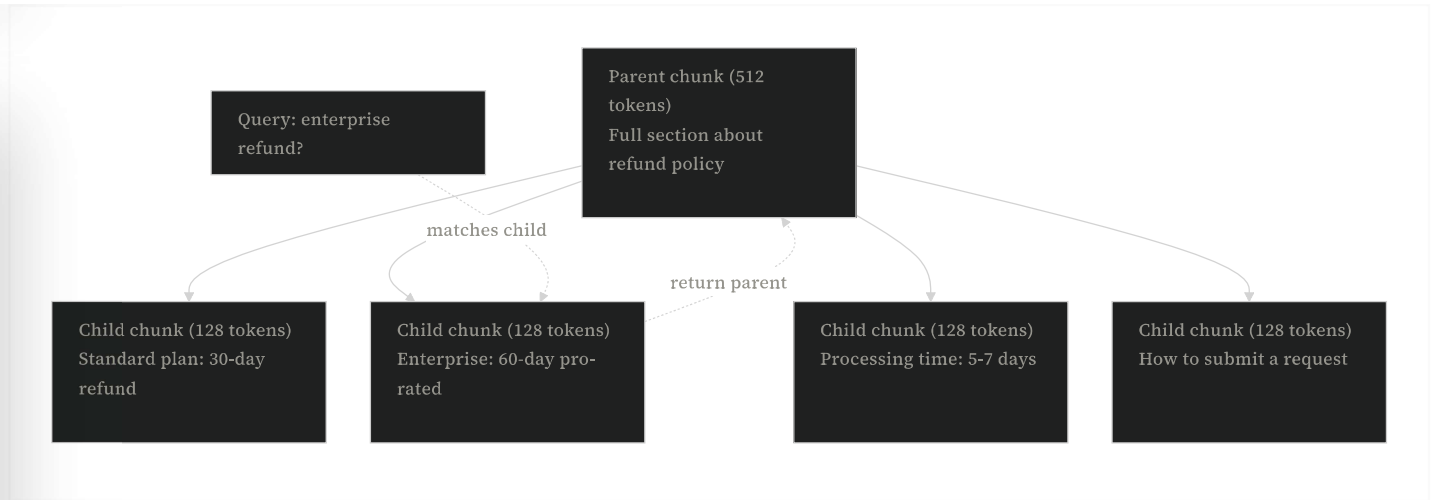
HyDE adds one LLM call before retrieval. This increases latency by 500-2000ms. Worth it when retrieval quality is poor on raw queries.

PARENT-CHILD CHUNKING

Standard chunking forces a trade-off: small chunks for precise retrieval, large chunks for sufficient context. Parent-child chunking eliminates this trade-off.

Index small chunks (128 tokens) for retrieval. When a small chunk is retrieved, return its parent chunk (512 tokens) for the prompt. The small chunk matches the query precisely. The parent chunk provides enough context for the LLM to generate a good answer.

EXPAND



The query "enterprise refund?" matches child chunk C2 precisely. But the prompt receives the full parent chunk P, which includes the surrounding context about processing time and submission process.

METADATA FILTERING

Before running vector search, filter the corpus by metadata: date, source, category, author, language. This reduces the search space and prevents irrelevant results.

"What changed in the security policy last month?" should only search documents from the last 30 days in the security category. Without metadata filtering, you search the entire corpus and might retrieve a 2-year-old security document that happens to be semantically similar.

Production RAG systems store metadata alongside each chunk: source document, creation date, category, author, version. Vector databases support pre-filtering by metadata before similarity search, which is critical for performance at scale.

EVALUATION

You built a RAG system. How do you know if it works? Three metrics:

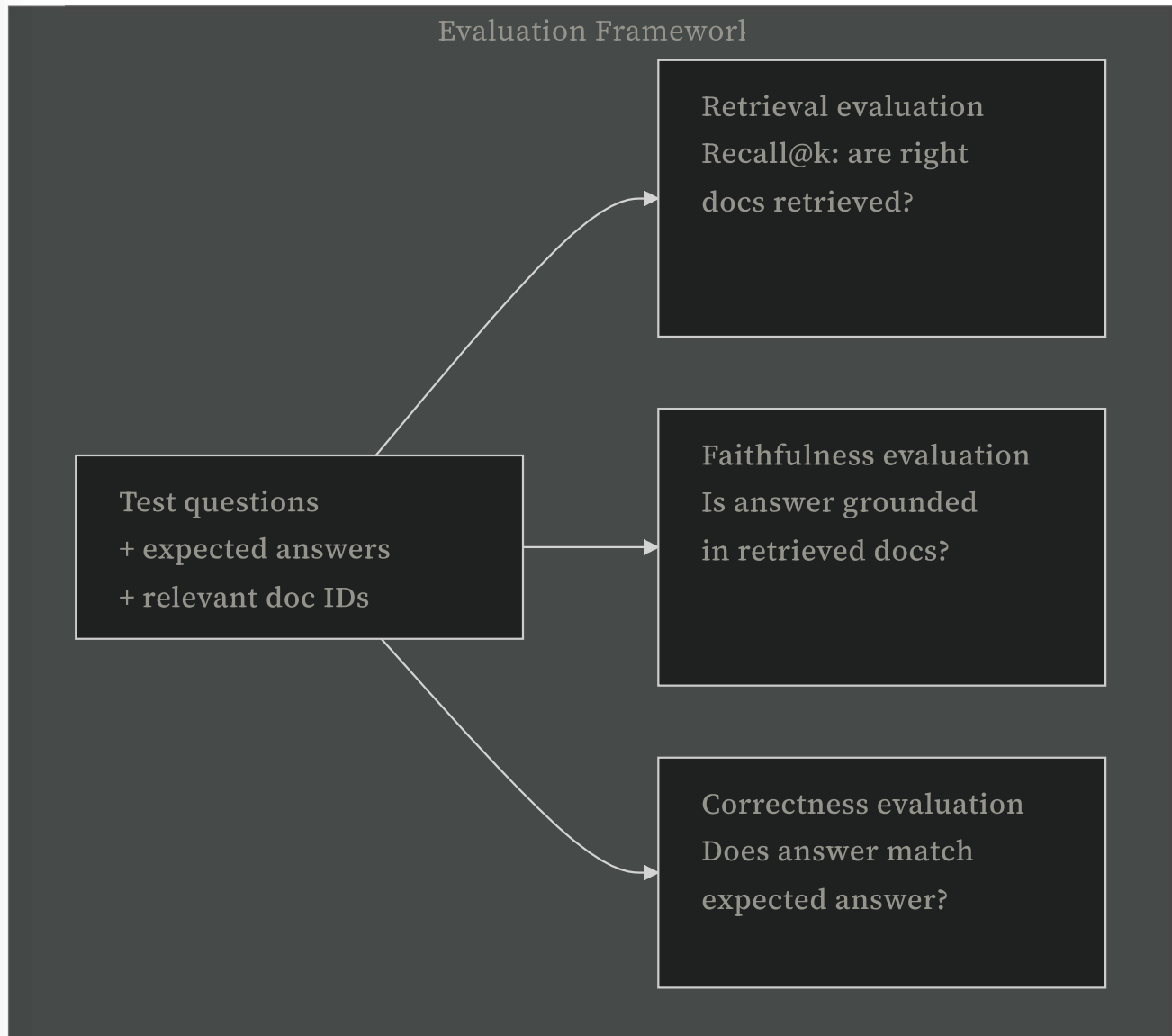
Retrieval relevance (Recall@k): for a set of test questions with known relevant documents, what percentage of relevant documents appear in the top-k results? If the answer to a question is in chunk #47, does chunk #47 appear in the top-5?

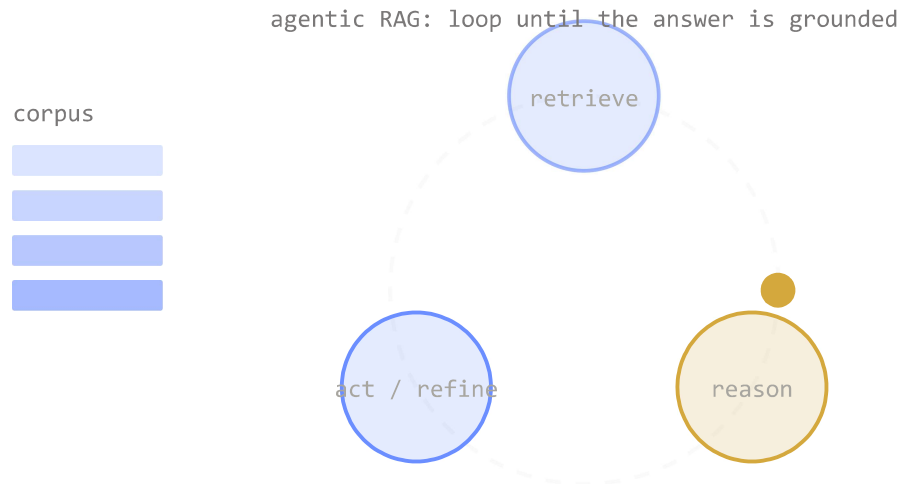
Faithfulness: is the generated answer grounded in the retrieved documents? If the retrieved chunks say "60-day refund window" and the model says "90-day refund window," that is a faithfulness failure. The model hallucinated despite having the correct context.

Answer correctness: does the generated answer match the expected answer? This is the end-to-end metric. It combines retrieval quality and generation quality.

A simple faithfulness check: take each claim in the generated answer and verify it appears (in substance) in the retrieved chunks. If the answer contains a fact not in any retrieved chunk, it is likely hallucinated.

EXPAND





Basic RAG retrieves once and answers. Agentic RAG loops: retrieve candidates, reason over whether they actually answer the query, then act — rewrite the query, rerank, or retrieve again. The orbiting token is one query cycling the loop until the context is good enough to answer, which is what multi-hop questions need.

BUILD IT

STEP 1: BM25 IMPLEMENTATION

PYTHON

COPY

```

import math
from collections import Counter

class BM25:
    def __init__(self, k1=1.2, b=0.75):
        self.k1 = k1
        self.b = b
        self.docs = []
        self.doc_lengths = []
        self.avg_d1 = 0
        self.doc_freqs = {}
        self.n_docs = 0
  
```

```

def index(self, documents):
    self.docs = documents
    self.n_docs = len(documents)
    self.doc_lengths = []
    self.doc_freqs = {}

    for doc in documents:
        words = doc.lower().split()
        self.doc_lengths.append(len(words))
        unique_words = set(words)
        for word in unique_words:
            self.doc_freqs[word] = self.doc_freqs.get(word, 0) + 1

    self.avg_dl = sum(self.doc_lengths) / self.n_docs if self.n_docs else 1

def score(self, query, doc_idx):
    query_words = query.lower().split()
    doc_words = self.docs[doc_idx].lower().split()
    doc_len = self.doc_lengths[doc_idx]
    word_counts = Counter(doc_words)
    score = 0.0

    for term in query_words:
        if term not in word_counts:
            continue
        tf = word_counts[term]
        df = self.doc_freqs.get(term, 0)
        idf = math.log((self.n_docs - df + 0.5) / (df + 0.5) + 1)
        numerator = tf * (self.k1 + 1)
        denominator = tf + self.k1 * (1 - self.b + self.b * doc_len / self.avg_dl)
        score += idf * numerator / denominator

    return score

def search(self, query, top_k=10):
    scores = [(i, self.score(query, i)) for i in range(self.n_docs)]
    scores.sort(key=lambda x: x[1], reverse=True)
    return scores[:top_k]

```

STEP 2: RECIPROCAL RANK FUSION

PYTHON

COPY

```

def reciprocal_rank_fusion(ranked_lists, k=60):
    scores = {}
    for ranked_list in ranked_lists:
        for rank, (doc_id, _) in enumerate(ranked_list):

```

```

    if doc_id not in scores:
        scores[doc_id] = 0.0
    scores[doc_id] += 1.0 / (k + rank + 1)
fused = sorted(scores.items(), key=lambda x: x[1], reverse=True)
return fused

```

STEP 3: HYBRID SEARCH PIPELINE

PYTHON

COPY

```

def hybrid_search(query, chunks, vector_embeddings, vocab, idf, bm25_index, top_k=5, fusion_k=3):
    query_emb = tfidf_embed(query, vocab, idf)
    vector_results = search(query_emb, vector_embeddings, top_k=top_k * 3)
    bm25_results = bm25_index.search(query, top_k=top_k * 3)
    fused = reciprocal_rank_fusion([vector_results, bm25_results], k=fusion_k)
    return fused[:top_k]

```

STEP 4: SIMPLE RERANKER

In production, you would use a cross-encoder model. Here we build a reranker that scores query-document relevance using word overlap, term importance, and phrase matching.

PYTHON

COPY

```

def rerank(query, candidates, chunks):
    query_words = set(query.lower().split())
    stop_words = {"the", "a", "an", "is", "are", "was", "were", "what", "how",
                  "why", "when", "where", "do", "does", "for", "of", "in", "to",
                  "and", "or", "on", "at", "by", "it", "its", "this", "that",
                  "with", "from", "be", "has", "have", "had", "not", "but"}
    query_terms = query_words - stop_words

    scored = []
    for doc_id, initial_score in candidates:
        chunk = chunks[doc_id].lower()
        chunk_words = set(chunk.split())

        term_overlap = len(query_terms & chunk_words)

        query_bigrams = set()
        q_list = [w for w in query.lower().split() if w not in stop_words]
        for i in range(len(q_list) - 1):
            query_bigrams.add(q_list[i] + " " + q_list[i + 1])
        bigram_matches = sum(1 for bg in query_bigrams if bg in chunk)

```

```

position_boost = 0
for term in query_terms:
    pos = chunk.find(term)
    if pos != -1 and pos < len(chunk) // 3:
        position_boost += 0.5

rerank_score = (
    term_overlap * 1.0
    + bigram_matches * 2.0
    + position_boost
    + initial_score * 5.0
)
scored.append((doc_id, rerank_score))

scored.sort(key=lambda x: x[1], reverse=True)
return scored

```

STEP 5: HYDE (HYPOTHETICAL DOCUMENT EMBEDDINGS)

PYTHON

COPY

```

def hyde_generate_hypothesis(query):
    templates = {
        "what": "The answer to '{query}' is as follows: Based on our documentation, {topic} i
        "how": "To address '{query}': The process involves several steps. First, you need to
        "default": "Regarding '{query}': Our records indicate specific details and policies r
    }
    query_lower = query.lower()
    if query_lower.startswith("what"):
        template = templates["what"]
    elif query_lower.startswith("how"):
        template = templates["how"]
    else:
        template = templates["default"]

    topic_words = [w for w in query_lower.split()
                    if w not in {"what", "is", "the", "how", "do", "does", "a", "an",
                                "for", "of", "to", "in", "on", "at", "by", "and", "or"}]
    topic = " ".join(topic_words) if topic_words else "this topic"

    return template.format(query=query, topic=topic)

def hyde_search(query, chunks, vector_embeddings, vocab, idf, top_k=5):
    hypothesis = hyde_generate_hypothesis(query)
    hypothesis_emb = tfidf_embed(hypothesis, vocab, idf)

```

```
results = search(hypothesis_emb, vector_embeddings, top_k)
return results, hypothesis
```

STEP 6: PARENT-CHILD CHUNKING

PYTHON

COPY

```
def create_parent_child_chunks(text, parent_size=200, child_size=50):
    words = text.split()
    parents = []
    children = []
    child_to_parent = {}

    parent_idx = 0
    start = 0
    while start < len(words):
        parent_end = min(start + parent_size, len(words))
        parent_text = " ".join(words[start:parent_end])
        parents.append(parent_text)

        child_start = start
        while child_start < parent_end:
            child_end = min(child_start + child_size, parent_end)
            child_text = " ".join(words[child_start:child_end])
            child_idx = len(children)
            children.append(child_text)
            child_to_parent[child_idx] = parent_idx
            child_start += child_size

        parent_idx += 1
        start += parent_size

    return parents, children, child_to_parent
```

STEP 7: FAITHFULNESS EVALUATION

PYTHON

COPY

```
def evaluate_faithfulness(answer, retrieved_chunks):
    answer_sentences = [s.strip() for s in answer.split(".")]
    if not answer_sentences:
        return 1.0, []

    grounded = 0
```

```

ungrounded = []
context = " ".join(retrieved_chunks).lower()

for sentence in answer_sentences:
    words = set(sentence.lower().split())
    stop_words = {"the", "a", "an", "is", "are", "was", "were", "and", "or",
                  "to", "of", "in", "for", "on", "at", "by", "it", "this", "that"}
    content_words = words - stop_words
    if not content_words:
        grounded += 1
        continue

    matched = sum(1 for w in content_words if w in context)
    ratio = matched / len(content_words) if content_words else 0

    if ratio ≥ 0.5:
        grounded += 1
    else:
        ungrounded.append(sentence)

score = grounded / len(answer_sentences) if answer_sentences else 1.0
return score, ungrounded

```

```

def evaluate_retrieval_recall(queries_with_relevant, retrieval_fn, k=5):
    total_recall = 0.0
    results = []

    for query, relevant_indices in queries_with_relevant:
        retrieved = retrieval_fn(query, k)
        retrieved_indices = set(idx for idx, _ in retrieved)
        relevant_set = set(relevant_indices)
        hits = len(retrieved_indices & relevant_set)
        recall = hits / len(relevant_set) if relevant_set else 1.0
        total_recall += recall
        results.append({
            "query": query,
            "recall": recall,
            "hits": hits,
            "total_relevant": len(relevant_set)
        })

    avg_recall = total_recall / len(queries_with_relevant) if queries_with_relevant else 0
    return avg_recall, results

```

USE IT

With a real cross-encoder for reranking:

PYTHON

COPY

```
from sentence_transformers import CrossEncoder

reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

def rerank_with_cross_encoder(query, candidates, chunks, top_k=5):
    pairs = [(query, chunks[doc_id]) for doc_id, _ in candidates]
    scores = reranker.predict(pairs)
    scored = list(zip([doc_id for doc_id, _ in candidates], scores))
    scored.sort(key=lambda x: x[1], reverse=True)
    return scored[:top_k]
```

With Cohere's managed reranker:

PYTHON

COPY

```
import cohere

co = cohere.Client()

def rerank_with_cohere(query, candidates, chunks, top_k=5):
    docs = [chunks[doc_id] for doc_id, _ in candidates]
    response = co.rerank(
        model="rerank-english-v3.0",
        query=query,
        documents=docs,
        top_n=top_k
    )
    return [(candidates[r.index][0], r.relevance_score) for r in response.results]
```

For HyDE with a real LLM:

PYTHON

COPY

```
import anthropic

client = anthropic.Anthropic()

def hyde_with_llm(query):
    response = client.messages.create(
```

```
model="claude-sonnet-5",
max_tokens=256,
messages=[{
    "role": "user",
    "content": f"Write a short paragraph that would be a good answer to this question"
}]
)
return response.content[0].text
```

For production hybrid search with Weaviate:

PYTHON

COPY

```
import weaviate

client = weaviate.connect_to_local()

collection = client.collections.get("Documents")
response = collection.query.hybrid(
    query="enterprise refund policy",
    alpha=0.5,
    limit=10
)
```

The alpha parameter controls the balance: 0.0 = pure keyword (BM25), 1.0 = pure vector, 0.5 = equal weight. Most production systems use alpha between 0.3 and 0.7.

SHIP IT

This lesson produces:

`outputs/prompt-advanced-rag-debugger.md` -- a prompt for diagnosing and fixing RAG quality issues

`outputs/skill-advanced-rag.md` -- a skill for building production-grade RAG with hybrid search and reranking

EXERCISES

- 01 Compare BM25 vs vector search vs hybrid search on the sample documents. For each of the 5 test queries, record which approach returns the most relevant chunk in position #1. Hybrid search should win on at least 3 out of 5.
- 01 Implement a metadata filter. Add a "category" field to each document (security, billing, api, product). Before running vector search, filter chunks to only the relevant category. Test with "What encryption is used?" and verify it only searches security-category chunks.
- 01 Build a full HyDE pipeline using the simple generate function from Lesson 06. Compare retrieval quality (top-3 relevance) between direct query search and HyDE search on all 5 test queries. HyDE should improve results for vague queries.
- 01 Implement the parent-child chunking strategy on the sample documents. Use child_size=30 and parent_size=100. Search with child chunks but return parent chunks in the prompt. Compare the generated answers to standard chunking with chunk_size=50.
- 01 Create an evaluation dataset: 10 questions with known answer chunks. Measure Recall@3, Recall@5, and Recall@10 for (a) vector search only, (b) BM25 only, (c) hybrid search, (d) hybrid + reranking. Plot the results and identify where reranking helps most.

✔ POST-LESSON QUIZ

1. What does a cross-encoder reranker do in an advanced RAG pipeline?

- ☐ It encodes documents into vectors
- ☐ It generates the final answer
- ☐ It splits documents into chunks
- ☐ It takes (query, document) pairs and scores their relevance with higher accuracy than embedding similarity, reordering the initial retrieval results

2. What is the HyDE (Hypothetical Document Embedding) query transformation technique?

- ☐ Encrypting the query for privacy

- ☐ Using the LLM to generate a hypothetical answer, then embedding that answer as the search query instead of the original question
- ☐ Expanding abbreviations in the query
- ☐ Hiding the query from the model

3. Why does parent-child chunking improve RAG over flat chunking?

- ☐ It's faster to index
- ☐ Small child chunks are used for precise retrieval, but the larger parent chunk is returned for context, preventing the 'lost context' problem
- ☐ It reduces the number of chunks
- ☐ It eliminates the need for embeddings

KEY TERMS

TERM	WHAT PEOPLE SAY	WHAT IT ACTUALLY MEANS
BM25	"Keyword search"	A probabilistic ranking algorithm that scores documents
Hybrid search	"Best of both worlds"	Running semantic (vector) and keyword (BM25) search
Reciprocal Rank Fusion	"Merge ranked lists"	Combining multiple ranked lists by summing 1/(rank + 1)
Reranking	"Second pass scoring"	Using a more expensive cross-encoder model to re-rank results
Cross-encoder	"Joint query-document model"	A model that takes a query and document as a single input
Bi-encoder	"Independent embedding model"	A model that embeds queries and documents independently
HyDE	"Search with a fake answer"	Generate a hypothetical answer to the query, embed it, and search
Parent-child chunking	"Small search, big context"	Index small chunks for precise retrieval but return the full parent chunk
Metadata filtering	"Narrow before searching"	Filtering documents by attributes (date, source, category) before search

TERM	WHAT PEOPLE SAY	WHAT IT ACTUALLY MEANS
Faithfulness	<i>"Did it stay grounded"</i>	Whether the generated answer is supported by th

FURTHER READING

Robertson & Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond" (2009) - the definitive reference for BM25, explaining the probabilistic foundations behind the formula

Cormack et al., "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods" (2009) -- the original RRF paper showing it beats more complex fusion methods

Gao et al., "Precise Zero-Shot Dense Retrieval without Relevance Labels" (2022) -- the HyDE paper demonstrating that hypothetical document embeddings improve retrieval without any training data

Nogueira & Cho, "Passage Re-ranking with BERT" (2019) -- showed cross-encoder reranking on top of BM25 significantly improves retrieval quality

[Khattab et al., "DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines" \(2023\)](#) -- treats prompt construction and weight selection as an optimization problem over retrieval pipelines; read this for "program LLMs" instead of "prompt LLMs."

[Edge et al., "From Local to Global: A Graph RAG Approach to Query-Focused Summarization" \(Microsoft Research 2024\)](#) -- GraphRAG paper: entity-relation extraction + Leiden community detection for query-focused summarization; the global vs local retrieval distinction.

[Asai et al., "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection" \(ICLR 2024\)](#) -- self-evaluating RAG with reflection tokens; the agentic frontier past static retrieve-then-generate.

[LangChain Query Construction blog](#) -- how to translate natural-language queries into structured database queries (Text-to-SQL, Cypher) as a pre-retrieval step.

0

WHAT THIS LESSON SHIPS

Prompts, skills, and artifacts you can use right now

PROMPT

prompt-advanced-rag-debugger

Diagnose and fix RAG quality issues across retrieval, generation, and evaluation

OPEN FILE

INSTALL

SKILL

skill-advanced-rag

Build production-grade RAG with hybrid search, reranking, and evaluation

OPEN FILE

INSTALL

COPY INSTALL

C

RUN THE CODE

Requires a local clone. Run copied commands from the repository root, the directory containing README.md and phases/.

PY main.py

22.9 KB

```
python3 phases/11-llm-engineering/07-advanced-rag/code/main.py
```

OPEN FILE

COPY COMMAND

Q

TEST YOUR UNDERSTANDING

Did you get it?

QUESTION 1 OF 3

What does a cross-encoder reranker do in an advanced RAG pipeline?

A It encodes documents into vectors

B It generates the final answer

- ☐ C It splits documents into chunks
- ☐ D It takes (query, document) pairs and scores their relevance with higher accuracy than embedding similarity, reordering the initial retrieval results

QUESTION 2 OF 3

What is the HyDE (Hypothetical Document Embedding) query transformation technique?

- ☐ A Encrypting the query for privacy
- ☐ B Using the LLM to generate a hypothetical answer, then embedding that answer as the search query instead of the original question
- ☐ C Expanding abbreviations in the query
- ☐ D Hiding the query from the model

QUESTION 3 OF 3

Why does parent-child chunking improve RAG over flat chunking?

- ☐ A It's faster to index
- ☐ B Small child chunks are used for precise retrieval, but the larger parent chunk is returned for context, preventing the 'lost context' problem
- ☐ C It reduces the number of chunks
- ☐ D It eliminates the need for embeddings

Want a deeper quiz? In Codex use `check-understanding 11` or choose it from `/skills` . In Claude Code use `/check-understanding 11` . In another compatible host say: `Use check-understanding to quiz me on Phase 11.`

AI Data Systems

Lesson 8 of 11

6 earlier

☐ 07 RAG: Retrieval-Augmented Generation

☒ 08 **Advanced RAG: Chunking, Reranking**

☐ 09 Evaluation & Testing

2 later

0 of 11 lessons completed in this path.

[← PREVIOUS](#)

RAG: Retrieval-Augmented Generation

[NEXT →](#)

Evaluation & Testing